

◆ CS EDUCATION ◆

🔍

소셜 로그인

26.07.03
배선아

교육 목표

1. 글로벌 인증 표준 : OAuth2.0
2. 무상태 웹 아키텍처
3. 데이터 정합성 관리
4. 토큰 만료화 정책

유저 이탈을 부르는 번거로운 가입 절차

[캡차 지옥]



[비밀번호 규칙]

NAVER

아이디

@naver.com

비밀번호

..... 사용불가 🔒

8~16자 영문 대 소문자, 숫자, 특수문자를 사용하세요.

이름

생년월일

[문자 인증]

PASS

PASS로 인증하기

문자(SMS)로 인증

이름

성명입력

생년월일/성별

.....

휴대폰번호

-없이 숫자만 입력

보안문자 입력

448608

보안문자 입력

취소

확인

이용약관 | 개인정보처리방침

256 bit 암호화 통신 적용

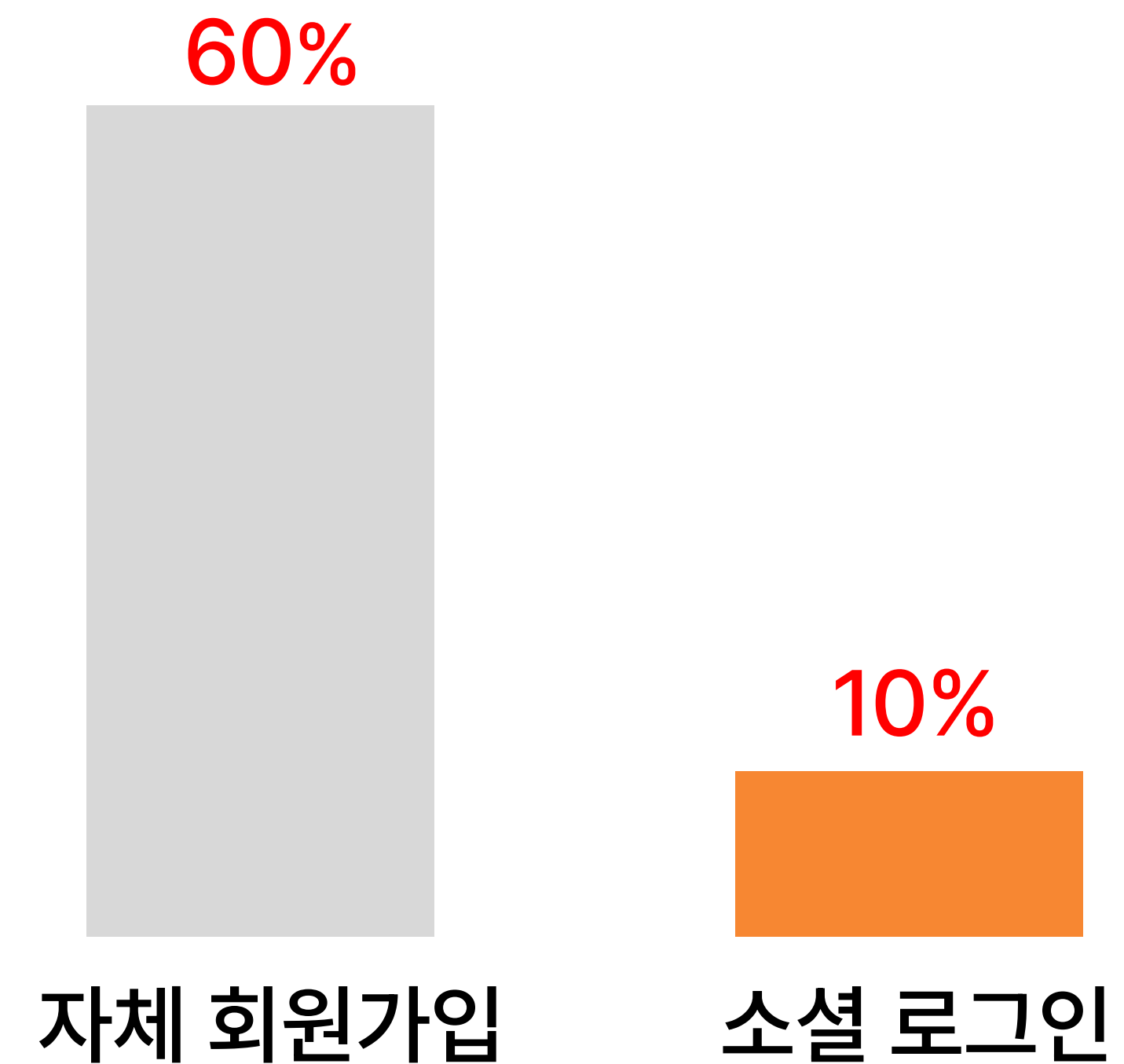
(주)드림시큐리티

소셜 로그인이 가져온 이탈률의 변화



“단 몇 번의 터치”

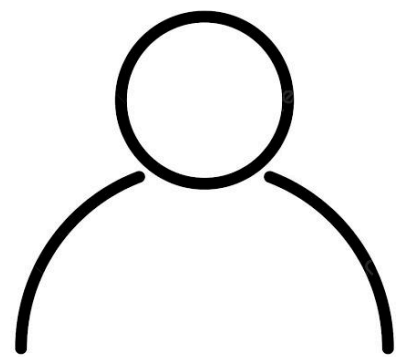
이탈률 변화



*출처: Baymard Institute / Gigya UX Research

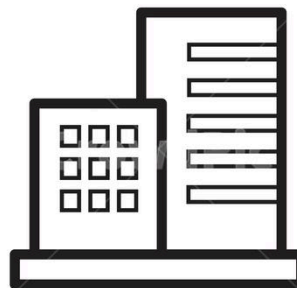
소셜 로그인이란?

카카오, 구글, 네이버 같은 대형 플랫폼의 계정을 이용해
다른 서비스에 로그인하는 방식



나(유저)

로그인 하고 싶어요!



Google / Kakao

제 계정 정보 드릴게요

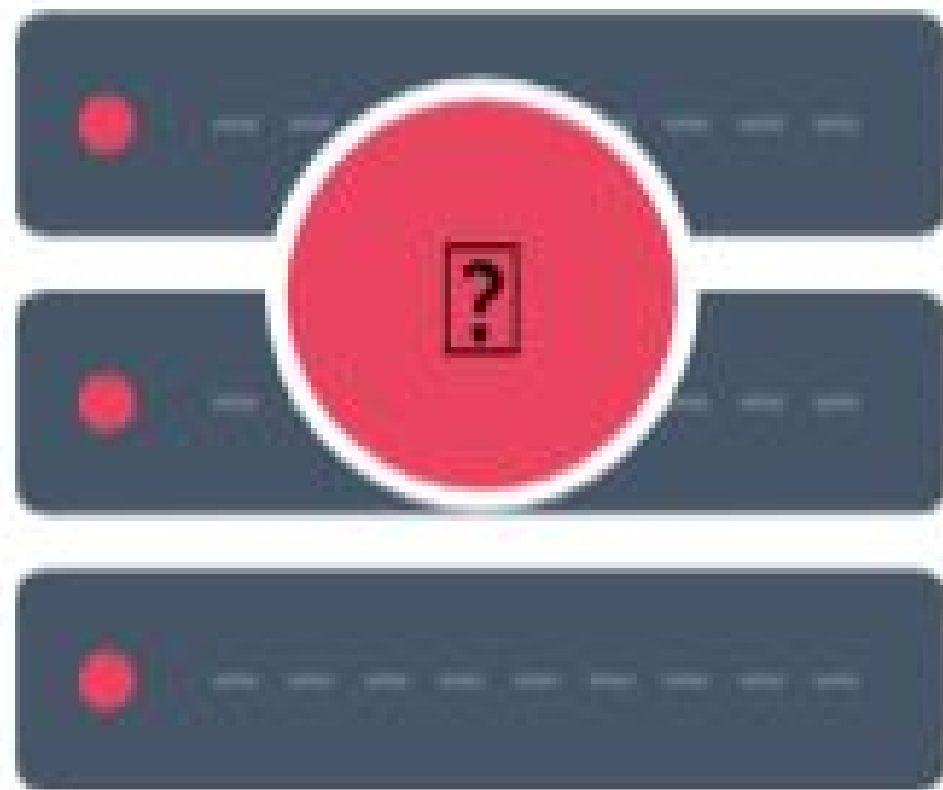


웹 / 서비스

로그인 완료!

왜 대기업에 확인을 맡길까?

자체 회원가입 리스크



- 비밀번호 암호화 직접 구현
- 해킹 시 전체 데이터 유출 위험

대기업 보안 인프라 활용



VS

- 글로벌 기업의 최신 보안 기술 그대로 사용
- 보안 전담팀이 24시간 모니터링

인증 위임 표준 프로토콜: OAuth 2.0



OAuth = Open Authorization

전 세계 IT 기업들이 약속한 외부 기관 신분 확인 표준

OAuth 2.0의 동작 원리 (1)

[1] 로그인 버튼 클릭

유저 → 우리 서비스

유저가 서비스에서
'구글로 로그인' 버튼 클릭



[2] 구글로 이동

우리 서비스 → 구글

우리 서비스 유저를
구글 로그인 페이지로
이동시킴



[3] 구글에서 허락

구글 → 유저

구글이 유저에게
'이 서비스에 접근 권한을
넘겨줘도 될까요?' 물어봄

OAuth 2.0의 동작 원리 (2)

[4] 토큰 발급

구글 → 우리 서비스

→
유저가 동의하면
구글이 '통행증(토큰)'을
발급해 통행증으로
정보 조회 가능

[5] 정보 가져오기

우리 서비스 → 구글

→
우리 서비스 토큰으로
구글에서 이름, 이메일 등
필요한 정보만 가져옴

로그인 완료! 계정이 자동으로 생성되고 바로 서비스 이용 가능

매번 신분증을 보여줄 순 없다

 유저가 페이지를 이동할 때마다

다시 구글 로그인 창 호출

→ 토큰 발급

→ 정보 가져오기 무한 반복?

최악의 서비스 유저 경험(UX) 유발 / 서버 간 불필요한 네트워크 과부하

최초 인증은 한 번만! 로그인을 유지해 줄 웹 아키텍처가 필요

로그인 유지하기 : 세션(Stateful) vs 토큰(Stateless)

Stateful (세션 기반 인증)

- 서버 메모리(Session)에 유저 로그인 상태 저장
- 유저가 늘어날수록 서버 메모리 부담 가중

서버가 유저를 일일이 기억
→ 서버 바빠짐

Stateless (토큰 기반 인증)

- 서버는 유저의 상태를 전혀 저장하지 않음 (무상태)
- 데이터 자체에 인증 정보가 담긴 JWT 토큰 활용
- 대규모 트래픽 및 서버 확장에 매우 유리

서버가 유저를 기억하지 않음
→ 유저가 영수증을 들고 다님

디지털 자유이용권 팔찌, JWT



JWT (JSON Web Token)

로그인에 성공한 유저에게 채워주는 디지털 자유이용권 팔찌

← JWT 토큰 예시

요약 : OAuth 2.0과 JWT

OAuth 2.0 (문을 여는 절차)

비밀번호 리스크를 없애는 안전한 신분을 대행

JWT 토큰 (출입증)

서버 부담을 줄이는 암호화된 자유이용권 팔찌

진짜 서비스로 가는 길

“기능이 돌아가는 것과, 안전하고 친절한 서비스인 것은 다르다”

데이터 정합성을 보장하기 위한 예외 처리 전략

예외 상황 (1) : 필수 데이터 누락

선택적 동의 항목의 함정

- 유저는 네이버 로그인 시 '개인정보 수집 및 이용' 등 특정 권한을 거부 가능
- 서버에는 필수 필드가 Null인 상태로 데이터가 유입
- 이에 대한 예외 처리가 없다면 NullPointerException과 함께 시스템이 중단

“동작”만 구현했을 때 발생하는 치명적 결함

NAVER 🌐 한국어 ▾

☒ 전체 동의하기

실명 인증된 아이디로 가입, 위치기반서비스 이용약관(선택), 이벤트 · 혜택 정보 수신(선택) 동의를 포함합니다.

☒ 필수 네이버 이용약관 보기

☒ 선택 실명 인증된 아이디로 가입

☒ 선택 위치기반서비스 이용약관 보기

☒ 선택 개인정보 수집 및 이용 보기

☒ 광고성 정보 수신동의

• 개인정보 수집 및 이용 안내 ^

[어린이용 안내](#)

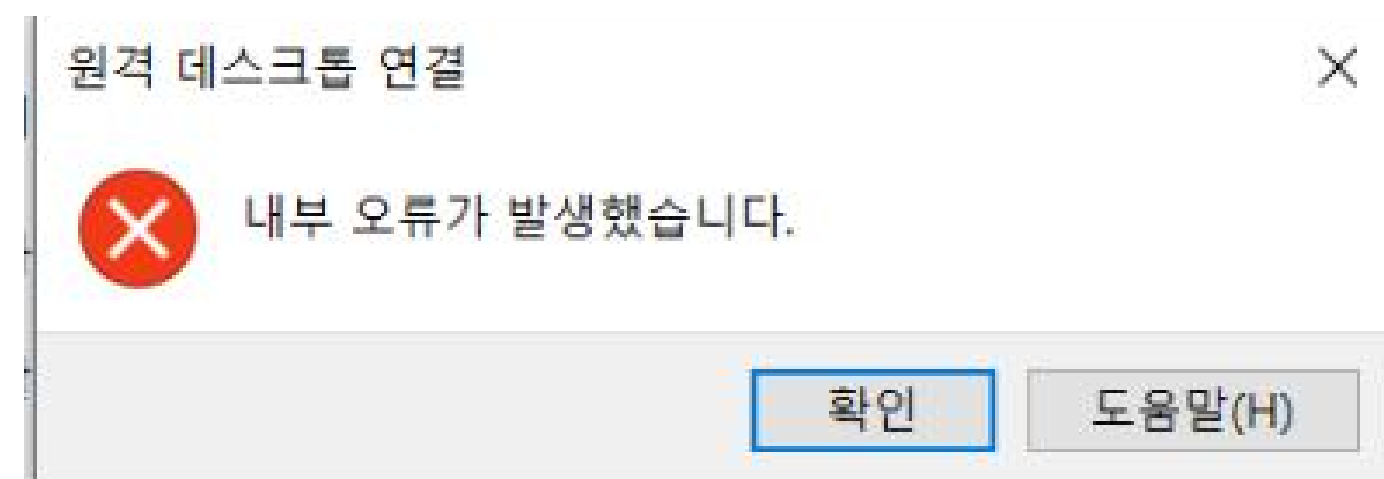
예외 핸들링과 서비스 UX

X 나쁜 예 (Raw Error)

시스템 에러 메시지를 유저에게 그대로 노출하거나 가입을 무작정 차단하는 방식

- “내부 오류가 발생했습니다”
- 유저는 원인을 알 수 없으며 서비스 이탈

예시 사진 →

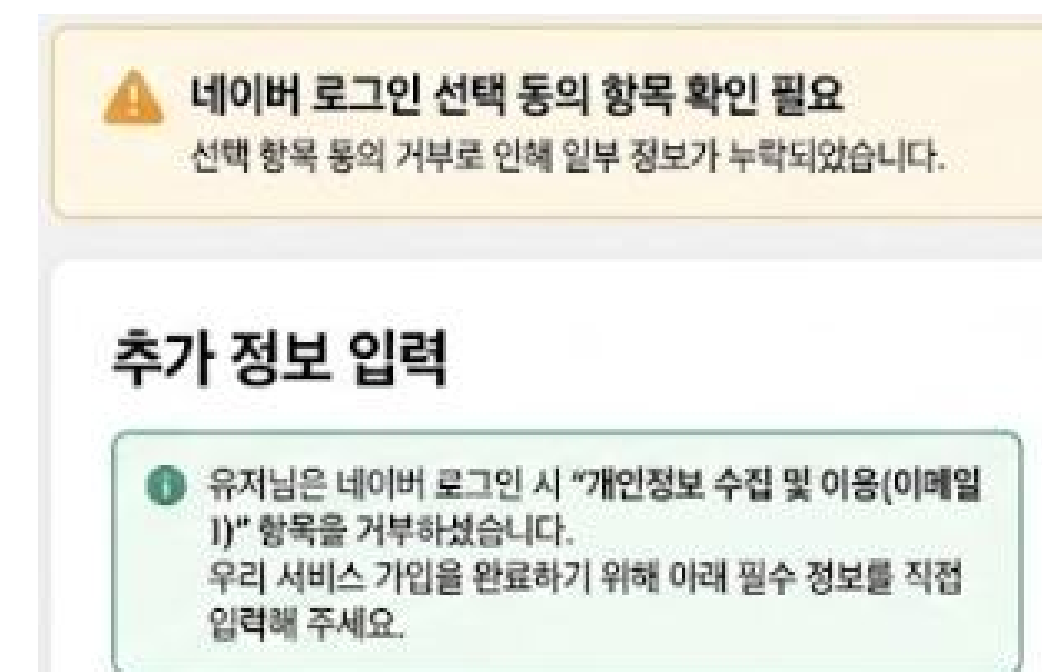


V 좋은 예 (Graceful Handling)

누락된 필수 데이터를 자연스럽게 수집하는 유연한 가입 흐름 설계

- ‘추가 입력 창’으로 유도하여 가입 유지
- 데이터 정합성과 유저 경험(UX)을 동시에 확보

예시 사진 →



예외 상황 (2) : 중복 데이터 충돌

계정 연동 아키텍처

기존 일반 가입 유저가 네이버 로그인을 중복 시도할 때 발생하는 식별자 충돌을 방지

- **식별자 체크:** 가입 전 이메일 중복 여부를 우선 확인
- **계정 연동(Linking):** “이미 가입된 계정이 있습니다. 연동하시겠습니까?” 팝업 제공
- **DB 정합성:** 하나의 유저ID 아래 여러 인증 수단(Local, Social) 매핑

기획-디자인-개발이 함께 풀어야 할 시스템 설계의 핵심



토큰 보관하기 : LocalStorage vs Cookie

LocalStorage (로컬 스토리지)

- 누구나 열어볼 수 있는 열립 서랍장
- 코드 한 줄이면 토큰을 꺼내 올 수 있음

→ 해킹 코드가 심어지면 토큰 노출 위험이 큼

Cookie (보안 쿠키)

- 자물쇠를 채워둔 철제 금고
- 브라우저에 숨겨져 있어서 코드로 접근 불가능

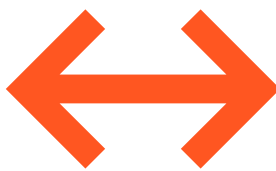
→ 해킹 코드가 들어와도 토큰을 절대 꺼낼 수 없음

보안성과 유저 경험(UX)의 시스템 딜레마

[보안 극대화]

토큰 유효기간을 10분으로 줄인다면?

- 토큰이 노출되도 10분 뒤면 무효
→ 해킹 위험 낮음
- 10분마다 강제 로그아웃
→ 유저 이탈



[편의성 극대화]

토큰 유효기간을 1년으로 늘린다면?

- 1년 동안 로그인 신경 안 써도 됨
→ 매우 편리
- 토큰 탈취 시 1년 내내 해킹 유입
→ 치명적 위험

안전하면서도 유저가 로그아웃되지 않는 방법은 없을까?

해결책 : 토큰 이중화 구조

Access Token (일반 자유이용권)

- 수명 : 30분 내외
- 역할 : 평소 모든 API 통신에 사용

Refresh Token (재발급 인증서)

- 수명 : 2주일
- 역할 : Access Token이 만료되었을 때 재발급용

[자동 갱신 흐름 요약]

Access Token 만료 → 서버가 Refresh Token 검증 → 새로운 Access Token 자동 발급

⇒ 유저는 로그아웃 없이 안전하게 서비스 지속 이용 가능

CS 키워드 요약

01

글로벌 인증 표준

OAuth 2.0 & JWT를
결합한 안전하고 효율적인
무상태 웹 아키텍처

02

데이터 정합성 관리

데이터 누락 및 중복 충돌을
방지하는 견고한 시스템 예외 처리

03

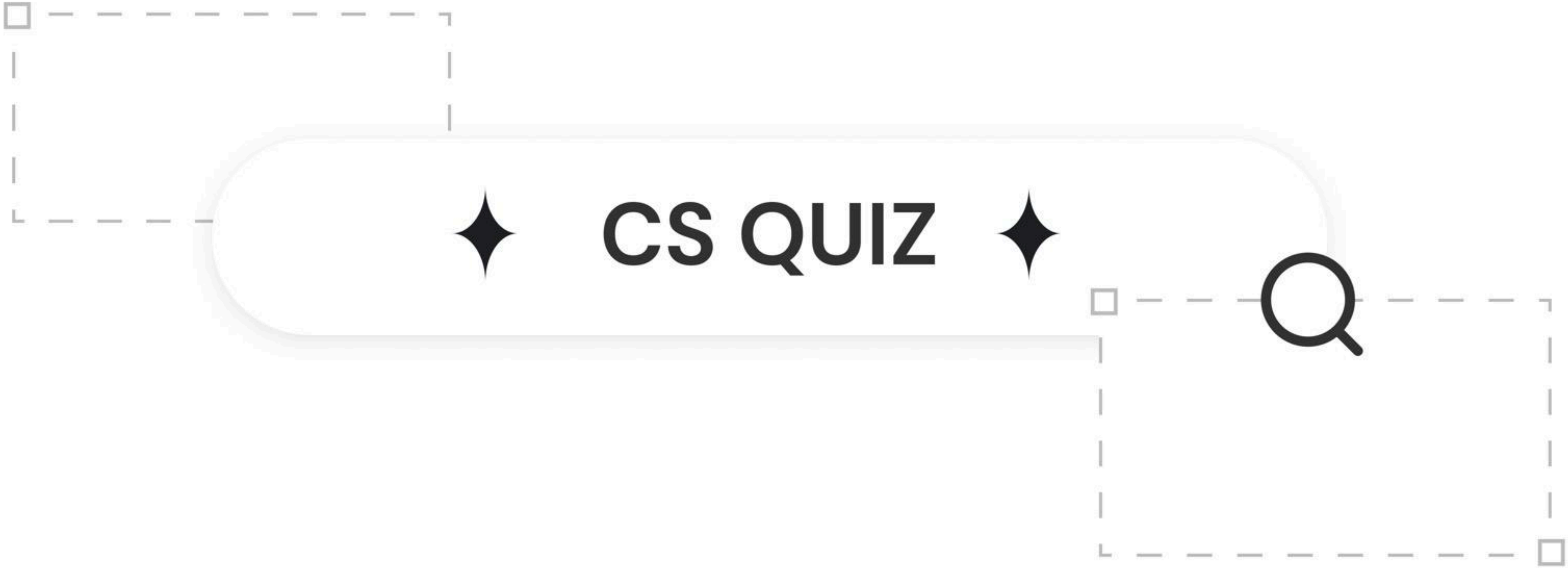
토큰 이중화 전략

Access & Refresh 토큰 분리로
보안과 유저 편의성(UX)을
동시에 만족

소셜 로그인 도입을 위한 실전 체크리스트

- ☐ 유저가 필수 데이터 제공을 거부했을 때의 공백 처리(추가 입력)을 설계했는가?
- ☐ 고유 식별자 충돌을 막기 위한 기존 중복 계정 연동 가이드와 UI를 챙겼는가?
- ☐ 보안성을 높이면서도 무중단 로그인을 보장할 토큰 저장/갱신 정책을 수립했는가?

이 3가지가 채워질 때 견고한 서비스가 완성됩니다



Q.1

OAuth 2.0의 가장 핵심적인 역할은 무엇일까요?

1. 사용자의 비밀번호를 우리 서버에 암호화해서 저장한다.
2. 사용자 인증을 외부 플랫폼(구글, 카카오 등)에 위임하고, 필요한 권한만 안전하게 전달받는다.
3. 로그인 상태를 서버 메모리에 저장한다.
4. JWT를 암호화하는 알고리즘이다.



A.1

OAuth 2.0의 가장 핵심적인 역할은 무엇일까요?

1. 사용자의 비밀번호를 우리 서버에 암호화해서 저장한다.
2. 사용자 인증을 외부 플랫폼(구글, 카카오 등)에 위임하고, 필요한 권한만 안전하게 전달받는다.
3. 로그인 상태를 서버 메모리에 저장한다.
4. JWT를 암호화하는 알고리즘이다.

OAuth 2.0은 비밀번호를 직접 받지 않고, 구글·카카오 같은 인증 제공자에게 사용자 인증을 맡기는 인증(Authorization) 표준 프로토콜입니다.

Q.2

다음 중 JWT에 대한 설명으로 맞는 것은 무엇일까요?

1. JWT의 Payload는 암호화되어 있어 아무도 볼 수 없다.
2. JWT는 Header, Payload, Signature의 세 부분으로 구성된다.
3. JWT는 서버가 모든 로그인 상태를 메모리에 저장해야 한다.
4. JWT는 OAuth 2.0을 사용할 때만 사용할 수 있다.



A.2

다음 중 JWT에 대한 설명으로 맞는 것은 무엇일까요?

1. JWT의 Payload는 암호화되어 있어 아무도 볼 수 없다.
2. **JWT는 Header, Payload, Signature의 세 부분으로 구성된다.**
3. JWT는 서버가 모든 로그인 상태를 메모리에 저장해야 한다.
4. JWT는 OAuth 2.0을 사용할 때만 사용할 수 있다.

JWT는 Header + Payload + Signature로 구성됩니다. Payload는 Base64로 인코딩되어 있을 뿐 누구나 확인할 수 있으므로 민감한 정보를 넣으면 안 됩니다.

Q.3

토큰 이중화를 사용하는 가장 큰 이유는 무엇일까요?

1. 로그인 속도를 빠르게 하기 위해
2. 데이터베이스 사용량을 줄이기 위해
3. 보안성과 사용자 경험(UX)을 동시에 확보하기 위해
4. JWT 크기를 줄이기 위해



A.3

토큰 이중화를 사용하는 가장 큰 이유는 무엇일까요?

1. 로그인 속도를 빠르게 하기 위해
2. 데이터베이스 사용량을 줄이기 위해
3. **보안성과 사용자 경험(UX)을 동시에 확보하기 위해**
4. JWT 크기를 줄이기 위해

Access Token은 짧게 사용해 탈취 위험을 줄이고, Refresh Token으로 새로운 Access Token을 발급받아 사용자는 다시 로그인하지 않아도 됩니다. 즉, 보안과 편의성을 동시에 만족시키기 위한 구조입니다.

Q.4

토큰 이중화 구조에서 평소 통신에 사용되며 수명이 짧은 토큰과,
만료 시 재발급용으로만 사용되며 수명이 긴 토큰을
각각 **(1) Token, (2) Token** 이라고 합니다.
다음 **(1), (2)**에 알맞은 용어를 쓰시오.

영어로 작성하시오 (대소문자 구분 X)



A.4

토큰 이중화 구조에서 평소 통신에 사용되며 수명이 짧은 토큰과,
만료 시 재발급용으로만 사용되며 수명이 긴 토큰을
각각 **(1) Token, (2) Token** 이라고 합니다.
다음 **(1), (2)**에 알맞은 용어를 쓰시오.

(1) : Access

(2) : Refresh

Q.5

다음 중 토큰 저장 방식에 대한 설명으로 가장 올바른 것은?

1. 로컬 스토리지는 코드로 접근이 불가능해 보안상 안전하다
2. 보안 쿠키는 자바스크립트 코드 한 줄로 쉽게 접근할 수 있어 편리하다
3. 로컬 스토리지는 자바스크립트로 접근이 가능해 토큰이 해킹당할 위험이 있다
4. 로컬 스토리지와 보안 쿠키는 보안 수준이 동일하므로 아무거나 사용해도 된다

A.5

다음 중 토큰 저장 방식에 대한 설명으로 가장 올바른 것은?

1. 로컬 스토리지는 코드로 접근이 불가능해 보안상 안전하다
2. 보안 쿠키는 자바스크립트 코드 한 줄로 쉽게 접근할 수 있어 편리하다
3. 로컬 스토리지는 자바스크립트로 접근이 가능해 토큰이 해킹당할 위험이 있다
4. 로컬 스토리지와 보안 쿠키는 보안 수준이 동일하므로 아무거나 사용해도 된다

로컬 스토리지는 누구나 열어볼 수 있는 서랍장 같아서 자바스크립트 코드 한 줄이면
토큰이 해킹당할 수 있다.

반면 보안 쿠키는 코드로 접근이 불가능한 철제 금고처럼 안전하기 때문에 보안 옵션을
적용한 보안 쿠키에 토큰을 저장하는 것이 안전한 설계의 기본이다.

Q.6

JWT의 '페이로드(Payload)' 부분은 서버의 비밀키로 암호화되어 있기 때문에, 해커가 중간에 가로채더라도 내용을 절대 열어볼 수 없다.

O/X 중 올바른 것은?



A.6

JWT의 '페이로드(Payload)' 부분은 서버의 비밀키로 암호화되어 있기 때문에, 해커가 중간에 가로채더라도 내용을 절대 열어볼 수 없다.
O/X 중 올바른 것은?

정답 : X

JWT의 페이로드는 암호화되어 있지 않고, 형태만 바꾼 '인코딩' 상태일 뿐입니다. 누구나 쉽게 열어볼 수 있는 '오픈된 신분증'과 같기 때문에 비밀번호나 개인정보 같은 민감한 데이터는 절대 넣으면 안 됩니다.



Q.7

유저의 로그인 상태를 유지하기 위해 사용되는 인증 방법 중, Stateless(토큰 기반 인증)의 특징이 아닌 것을 고르시오.

1. 유저가 늘어날수록 서버 메모리 부담이 가중된다.
2. 유저가 영수증을 들고 다니는 것과 같은 효과를 낸다.
3. 대규모 트래픽 및 서버 확장에 매우 유리하다.
4. 서버는 유저의 상태를 전혀 저장하지 않는다.



A.7

유저의 로그인 상태를 유지하기 위해 사용되는 인증 방법 중,
Stateless(토큰 기반 인증)의 특징이 아닌 것을 고르시오.

1. **유저가 늘어날수록 서버 메모리 부담이 가중된다.**
2. 유저가 영수증을 들고 다니는 것과 같은 효과를 낸다.
3. 대규모 트래픽 및 서버 확장에 매우 유리하다.
4. 서버는 유저의 상태를 전혀 저장하지 않는다.

**유저가 늘어날수록 서버 메모리 부담이 가중되는 것은
Stateful(세션 기반 인증)의 특징입니다.**



Q.8

다음 상황에서 유저 경험을 개선하기 위해 개발자가 취할 수 있는 좋은 예시를 고르시오.

<보기>

철웅이가 코테이토 홈페이지에 가입하려고 한다. 이때, 개인정보 수집 및 이용 권한 일부를 거부했더니 서비스가 멈추고 NullPointerException 라는 오류 코드가 화면에 떴다.

1. 누락된 필수 데이터를 서비스가 임의로 채워 넣는다.
2. 데이터베이스가 엉망이 될 수 있으니 일단 가입을 차단한다.
3. 필요한 데이터를 추가적으로 입력 받을 수 있는 화면으로 이용자를 유도한다.
4. 시스템 에러 메시지를 예쁘게 꾸며서 이용자에게 보여준다.

A.8

다음 상황에서 유저 경험을 개선하기 위해 개발자가 취할 수 있는 좋은 예시를 고르시오.

<보기>

철웅이가 코테이토 홈페이지에 가입하려고 한다. 이때, 개인정보 수집 및 이용 권한 일부를 거부했더니 서비스가 멈추고 NullPointerException 라는 오류 코드가 화면에 떴다.

3. 필요한 데이터를 추가적으로 입력 받을 수 있는 화면으로 이용자를 유도한다.

<보기>처럼 필수 데이터가 누락되었을 때 불친절한 에러 화면을 보여주면 유저는 이탈합니다. 따라서 예외를 핸들링하여 필요한 정보를 다시 입력받는 '추가 입력 창'으로 유도해야 합니다.

Q.9

이메일 중복 시 식별자 충돌을 해결하는 과정이다. 빈칸에 들어갈 알맞은 용어는?

이메일 중복이 확인되면 유저에게 (A) 팝업을 띄워 유도합니다.
이를 통해 하나의 유저 ID 아래 여러 인증 수단을 매핑함으로써 최종적으로
데이터의 (B)을(를) 확보할 수 있습니다.

1. (A) 계정 동기화 / (B) 데이터 가용성
2. (A) 식별자 매핑 / (B) DB 정규화
3. (A) 계정 연동 / (B) DB 정합성
4. (A) 유저 통합 / (B) 데이터 독립성

A.9

이메일 중복 시 식별자 충돌을 해결하는 과정이다. 빈칸에 들어갈 알맞은 용어는?

이메일 중복이 확인되면 유저에게 (A) 팝업을 띄워 유도합니다.
이를 통해 하나의 유저 ID 아래 여러 인증 수단을 매핑함으로써 최종적으로
데이터의 (B)을(를) 확보할 수 있습니다.

3. (A) 계정 연동 / (B) DB 정합성

중복 가입 시 유저에게는 UI로 '계정 연동'을 유도하고, 서버에서는 하나의 고유 ID에 여
러 인증 수단을 묶어 설계함으로써 데이터 유실 없이 'DB 정합성'을
유지해야 합니다.

Q.10

사용자가 '구글 로그인' 버튼을 누른 순간부터 로그인이 완료되기까지의 OAuth 2.0 동작 과정을 올바른 순서대로 나열하세요.

<보기>

- (A) 구글이 사용자에게 정보 공유 동의를 구하고, 사용자가 이에 동의한다.
- (B) 사용자가 구글 로그인 페이지에서 자신의 계정으로 직접 로그인한다.
- (C) 구글이 인증 코드(Authorization Code)를 발급하여 우리 서비스로 보낸다.
- (D) 사용자가 우리 서비스의 '구글 로그인' 버튼을 클릭하여 구글 로그인 페이지로 이동한다.
- (E) 우리 서비스가 발급받은 인증 코드를 가지고 구글 서버에 요청하여 진짜 사용자 정보(토큰)를 받아온다.

A.10

사용자가 '구글 로그인' 버튼을 누른 순간부터 로그인이 완료되기까지의 OAuth 2.0 동작 과정을 올바른 순서대로 나열하세요.

(D) → (B) → (A) → (C) → (E)

1. 로그인 버튼 클릭 및 페이지 이동 (D)
2. 구글 플랫폼에서 직접 로그인 (B)
3. 정보 제공 동의 (A)
4. 구글이 서비스에 인증 코드 전달 (C)
5. 서비스가 코드를 토큰으로 교환하며 로그인 완료 (E)